Practica 5A

Parte 1

Pruebas unitarias clase Seguro(precio())

```
/**
* Retorna el precio del seguro.
 * El precio se calcula a partir de la cobertura, la potencia del coche y
el tiempo que lleva contratado el seguro
 * @return El precio del seguro
 * 0 si el seguro todavía no está en vigor (no se ha alcanzado su fecha de
inicio)
*/
```

- fechaInicio: {null}, {futura}, {vigente < 1 año}, {exactamente 1 año, AVL}, {> 1 año}
- cobertura: {null}, {TERCEROS}, {TERCEROS_LUNAS}, {TODO_RIESGO}
- potencia: {< 90}, {[90, 110]}, {> 110}. Valores límite: 89, 90, 110, 111.

Método de prueba	fechaInicio	Potencia	Cobertura	Esperado
testFechaInicioNull	null	50	TERCEROS	0.0
testFechaInicioFutura	mañana	50	TERCEROS	0.0
testCoberturaNull	hace 2 años	50	null	0.0
testCoberturaTerceros	hace 2 años	15	TERCEROS	400.0
testCoberturaTercerosLunas	hace 2 años	25	TERC_LUNAS	600.0
testCoberturasTodoRiesgo	hace 2 años	20	TODO_RIESGO	1000.0
testPotencia89	hace 2 años	89	TERCEROS	400.0
testPotencia90	hace 2 años	90	TERCEROS	420.0
testPotencia110	hace 2 años	110	TERCEROS	420.0
testPotencia111	hace 2 años	111	TERCEROS	480.0
testPotenciaAlta	hace 2 años	300	TERCEROS	480.0
testDescuento20pct	hace 6 meses	15	TERCEROS	320.0
testSinDescuento	hace 1 año	15	TERCEROS	400.0

Pruebas unitarias clase Cliente totalSeguros()

```
/**
* Calcula el total a pagar por el cliente por
* todos los seguros a su nombre
*/
```

- minusvalia: {false} | {true}
- número de seguros: {0} | {1} | {2} | {3}
- vigencia de los seguros: {todos vigentes} | {alguno no vigente, precio = 0}

Método de prueba	Minusvalía	Seguros	Esperado
testTotalSeguros_SinSeguros	false	[]	0.0
testTotalSeguros_SinSeguros_ConMinusvalia	true	[]	0.0
testUnSeguro	false	2222AAA TERC_LUNAS pot=25	600.0
testUnSeguro_ConMinusvalia	true	2222AAA TERC_LUNAS pot=25	450.0
testDosSeguros	false	4444AAA + 4444BBB	1120.0
testSeguroNoVigente	false	1111AAA fechaInicio futura	0.0

Cliente Test

```
    public double totalSeguros() {
        double total = 0;
        for (Seguro s : seguros) {
            total += s.precio();
        }

        if (this.minusvalia) {
            total = total * 0.75;
        }
        return total;
    }
}
```

Seguro Test

```
    public double precio() {
        LocalDate ahora = LocalDate.now();
        if ( fechaInicio == null || fechaInicio.isAfter(ahora)) {
            return 0;
        }
        double precioB = 0;
        if (cobertura != null) {
            switch (cobertura) {
                case TODO_RIESGO:
                    precioB = 1000;
                    break;
                case TERCEROS_LUNAS:
                    precioB = 600;
                    break;
                case TERCEROS:
                    precioB = 400;
                    break;
            }
        }
```

Parte 2

Estado inicial base de datos

Cliente			Seguro				
DNI	Nombre	Minusvalia	Matricula	Cobertura	FechaInicio	Potencia	Conductor adicional
11111111A	Juan	false	1111AAA	Terceros	2002-01-15	15	
			1111BBB	Todo Riesgo	2016-05-20	20	Pepe
			1111CCC	Terceros	2022-05-21	100	
22222222A	Ana	False	2222AAA	Terceros Lunas	2010-06-01	25	
33333333A	Luis	True					
44444444A	Pepe	False	4444AAA	Terceros	2024-01-02	40	
			4444BBB	Terceros Lunas	2024-01-02	300	

ID	DNI / Escenario	Resultado esperado en UI	Nombre test
V01	11111111A (Juan, 3 seguros, sin minusvalía)	Nombre: Juan Total: 1820.0 Lista: 3 items	testSinMinusvalia
V02	99999999Z (no existe)	Nombre: Error en BBDD' Lista: 0 items	testClienteNoExistente
V03	Lanzo DataAccessException	Nombre: Error en BBDD Lista: 0 items	testConsultaCliente_FalloBBDD

Conclusión

Lo único que hemos necesitado de más reflexión ha sido el test FalloBBDD, donde tuvimos que simular una excepción sin framework de mocking, resolviéndolo con una subclase anónima que sobreescribía el método como sale en los apuntes. La caja blanca la descartamos al comprobar que los tests de caja negra ya llegaban a todos los caminos.